

PS-01: Multi-Agent Bug Detection & Auto PR System

Autonomous Software Maintenance via 10-Agent Orchestrated Pipeline

Agentic AI Hackathon 2026 - ASTRA Lab, IIT Madras - Level 2: Solution Architecture

Submitted by: Harsh Ganatra | harshganatra57346@gmail.com

1. Problem Understanding

PS-01 demands more than a scan-and-patch pipeline. It asks for a system that behaves like a senior engineer doing a complete audit: first building deep understanding of a repository before touching a single line, then discovering four distinct classes of issues (functional bugs, security vulnerabilities, code quality problems, and performance issues), then investigating root causes rather than symptoms, planning fixes in the right order, generating validated patches that actually pass tests, and finally producing structured pull requests with confidence scores and human approval gates for high-risk changes.

Four issue classes the system must detect

Class	Examples	Primary Detection Method
Functional Bugs	Null pointer exceptions, missing edge cases, incorrect conditions, broken APIs	LLM reasoning over file content + knowledge graph context
Security Vulnerabilities	SQL injection, XSS, CSRF, hardcoded secrets, weak authentication, JWT issues	Semgrep (p/security-audit, p/secrets) + Bandit + LLM confirmation
Code Quality	Dead code, duplicate code, long methods, circular dependencies	Static analysis (ESLint, pylint) + LLM pattern reasoning
Performance	Memory leaks, inefficient loops, N+1 database queries	LLM reasoning over ORM patterns and loop structures in knowledge graph

Why naive approaches fail

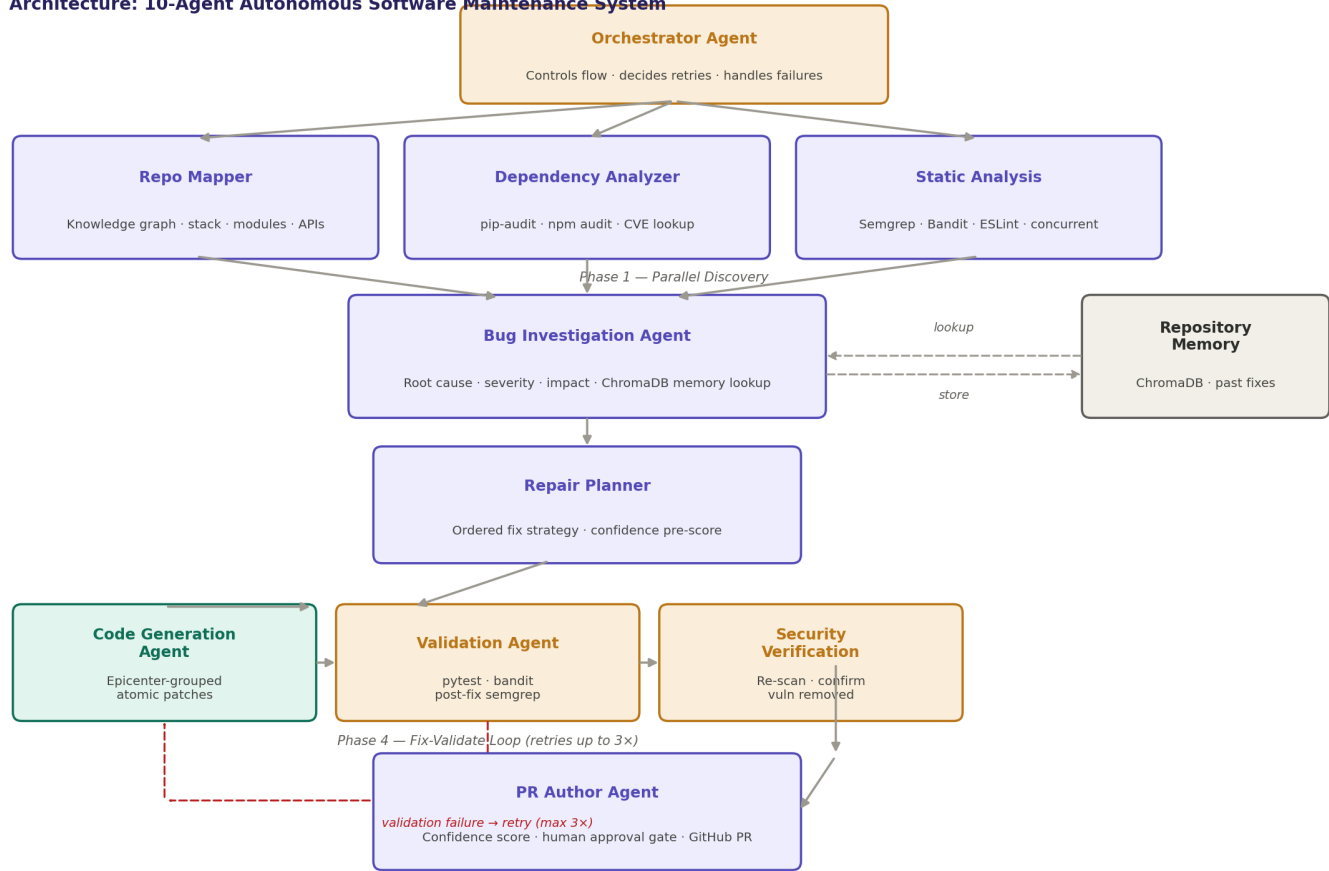
Existing tools - Patchwork, CodeRabbit, Semgrep Autofix - share a common failure: they treat each file in isolation. They cannot answer 'why does this bug exist?', 'which modules are affected?', or 'what is the safest fix order?' They also cannot verify that a fix actually works - they generate a patch and open a PR with no test execution. Our system addresses every one of these gaps through a ten-agent orchestrated pipeline with a real fix-validate feedback loop and semantic memory of past fixes.

In essence: an autonomous software maintenance agent that builds a knowledge graph of the repository, discovers all four issue classes, plans an ordered repair strategy, generates patches that pass actual tests, and publishes confidence-scored, human-approved pull requests.

2. Agent Architecture Design

Ten specialised agents organised into five phases, coordinated by an Orchestrator that controls sequencing, decides retries, and handles partial failures. Phase 1 runs in parallel (three agents concurrently). Phase 4 is a feedback loop: Code Gen - AST Validator - Security Verify, with retry arrows back to Code Gen on any failure.

Architecture: 10-Agent Autonomous Software Maintenance System



Agent	Phase	Core responsibility	Key tools
1. Repo Mapper	1 (parallel)	Build knowledge graph: language, framework, modules, API endpoints, DB interactions, service boundaries	ast module, grep, file tree walk
2. Dependency Analyzer	1 (parallel)	Parse requirements.txt / package.json / pom.xml; detect outdated and CVE-flagged packages	pip-audit, npm audit, safety
3. Static Analysis	1 (parallel)	Run all scanners concurrently; produce normalised findings list across all four issue classes	Semgrep, Bandit, ESLint
4. Bug Investigation	2	For each finding: determine root cause, severity, impact radius, affected modules; query ChromaDB for similar past fixes	Claude + 2-hop BFS on knowledge graph + ChromaDB
5. Repair Planner	3	Order fixes by dependency (fix A before B if A's fix affects B's file); generate repair plan; compute confidence pre-score	Claude + topological sort on affected file graph
6. Code Generation	4 (loop)	Generate atomic patch set grouped by epicenter file; all bugs in same file fixed in one LLM call	Claude (full file in context)
7. Validation Agent	4 (loop)	Run pytest, bandit, post-fix semgrep scan in sandboxed subprocess; on failure return error to Code Gen	pytest, bandit, semgrep, subprocess
8. Security Verification	4 (loop)	Re-run security scan to confirm original vulnerability is no longer present in patched files	Semgrep, Bandit targeted re-scan
9. PR Author	5	Compute final confidence score; apply human approval gate for high-risk changes; open GitHub PR with structured description	PyGithub, confidence scorer
10. Orchestrator	All	LLM-based controller: decides what runs next, when to retry, when to escalate, when to mark as unresolvable	Claude (reasoning over pipeline state)

3. Workflow & Agent Orchestration

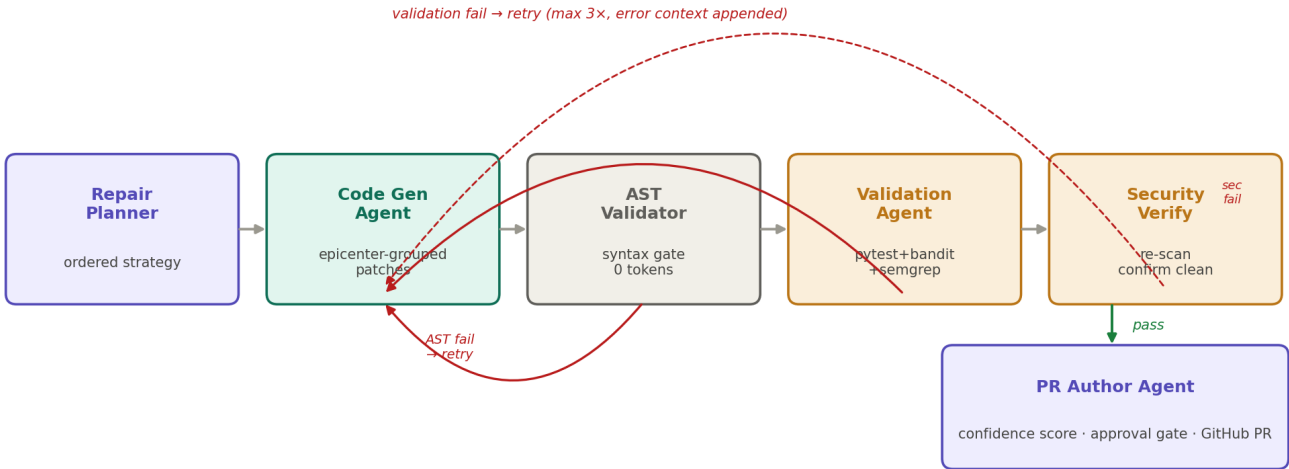
The Orchestrator Agent uses LLM reasoning over the current pipeline state to decide what runs next - not a hardcoded DAG. This means it can handle partial failures gracefully: if the Dependency Analyzer fails to reach the CVE registry, the Orchestrator skips that finding source and continues with Semgrep results rather than aborting the entire run. The three core orchestration patterns are described below.

Phase 1: Parallel discovery

Repo Mapper, Dependency Analyzer, and Static Analysis all receive the cloned repository path and run concurrently via `asyncio.gather`. None depends on the others. Semgrep typically takes 15-20 seconds; the knowledge graph build takes 2-3 seconds; both complete before the Orchestrator advances to Phase 2. This parallelism is the biggest single time-saver in the pipeline.

Phase 4: Fix-validate feedback loop

Phase 4: Fix-Validate Feedback Loop



Every patch set goes through four gates before a PR is opened. Gate 1 is a zero-cost AST syntax check (`ast.parse`) - catches malformed Python immediately with no API call. Gate 2 is the Validation Agent: it applies the patch to a sandboxed copy of the repo, runs `pytest` for the affected modules, runs `bandit` on patched files, and re-runs `semgrep` on the changed lines. Gate 3 is Security Verification: a targeted re-scan confirming the original vulnerability rule no longer fires. Only after all three gates pass does the Orchestrator advance to the PR Author. If any gate fails, the Orchestrator returns control to Code Gen with the full error output appended to the prompt, up to three retry attempts.

Repository Memory (ChromaDB)

Every successfully validated fix is stored in ChromaDB with four fields: the bug type, the root cause pattern, the fix strategy used, and the confidence score achieved. When the Bug Investigation agent encounters a new issue, it queries ChromaDB for semantically similar past fixes using the bug type and affected code snippet as the query vector. A cache hit boosts the initial confidence pre-score and provides the Repair Planner with a proven strategy to start from, reducing both LLM calls and retry attempts.

4. Technology Stack

Layer	Technology	Design decision
Orchestration	LangGraph StateGraph + Send API + Claude Orchestrator	LLM-based Orchestrator reasons over state rather than following hardcoded edges; handles partial failures adaptively
LLM	Claude Sonnet (Anthropic async client)	Used by: Orchestrator, Bug Investigation, Repair Planner, Code Gen, PR Author; 4-retry exp. backoff with jitter
Static analysis	Semgrep (p/security-audit, p/secrets) + Bandit + ESLint	Run concurrently in Phase 1; Semgrep + Bandit re-run in Phase 4 validation loop on patched files only
Dep. scanning	pip-audit + npm audit + safety	Detect CVE-flagged and outdated packages from requirements.txt / package.json; run in Phase 1 parallel
Test execution	pytest + subprocess sandbox	Validation Agent applies patch to temp copy; runs <code>pytest -x</code> on affected test modules; stdout captured for retry context
Repository Memory	ChromaDB (persistent, local)	Stores (bug_type, root_cause, fix_strategy, confidence) per resolved fix; queried by Bug Investigation via semantic similarity
Knowledge graph	Python ast + networkx	Directed graph of file-level imports, function defs, API routes, DB models; backbone of blast radius and repair ordering

Layer	Technology	Design decision
Source control	PyGithub + GitHub REST API	Async branch, commit, PR; confidence score and approval status embedded in PR description
Backend	FastAPI async + Pydantic BaseSettings	POST /run -> 202 + job_id; SSE stream at GET /jobs/{id}/stream; API-key auth middleware; per-IP rate limiting
Frontend	Vanilla JS + HTML	2-step job API; live agent feed; blast-radius bubble map; confidence score badge; human approval modal
Config	Pydantic BaseSettings	Validated at startup; fails loudly before serving if ANTHROPIC_API_KEY missing

5. Data Flow & Integrations

POST /run returns job_id (202) instantly; pipeline runs as asyncio background task. Phase 1 runs Repo Mapper (networkx graph), Dependency Analyzer (pip-audit + npm audit), and Static Analysis (Semgrep + Bandit + ESLint) concurrently. Bug Investigation queries ChromaDB then calls Claude with 10-line snippets only. Repair Planner topologically sorts fixes. Code Gen receives full file, patches pass AST gate then pytest + bandit + semgrep in sandbox, Security Verify re-runs targeted rules, PR Author applies confidence gate and opens GitHub PR. SSE streams every agent event to frontend.

6. Confidence Scoring

Signal	Weight	Rationale
pytest suite passes for affected modules	+40%	Strongest signal: behaviour preserved
Post-fix semgrep + bandit clean	+25%	Vulnerability confirmed removed
AST syntax valid (gate 1)	+10%	Structural correctness
ChromaDB cache hit (similar past fix succeeded)	+15%	Historical precedent
Repair Planner ordered this fix first (low dependency risk)	+10%	Low blast-radius change

A score below 70%, or any fix touching authentication, JWT, or cryptography paths, automatically triggers the **Human Approval Gate**: the PR is opened as a draft and an approval request is sent via the configured webhook (Slack or email). Above 70% with no critical path involvement, the PR is opened ready-to-merge and the team is notified.

7. Expected Outcomes

On a demo repo with four planted issues (SQL injection chain, hardcoded API key, N+1 ORM query, MD5 hash), the pipeline completes in under three minutes producing one PR per epicenter file, each with confidence score badge, blast-radius count, root-cause explanation, and human approval request where triggered.

8. Scalability & Advanced Features

- **Multi-repository mode**: the Orchestrator accepts a GitHub org pattern and queues one job per repo; ChromaDB memory is shared so fixes in one repo propagate as cache hits to similar issues in others.
- **CI/CD integration**: a GitHub webhook triggers a run on push to main or new PR; the Validation Agent reuses the repo's existing CI environment.
- **Knowledge graph persistence**: the networkx graph serialises to disk between runs; a second scan of the same repo skips Repo Mapper and goes straight to incremental diff analysis.
- **Parallel validation**: pytest, bandit, and semgrep run concurrently inside the Validation Agent; wall-clock time matches the slowest tool, not their sum.

9. Innovation vs Existing Tools

Capability	Semgrep AppSec (paid)	Patchwork AutoFix	This system
All 4 issue classes	Security only	Security only	Yes - functional, security, quality, perf
Dependency CVE scan	Paid add-on	No	Yes - pip-audit + npm audit, Phase 1
Knowledge graph	No	No	Yes - networkx, backbone of all reasoning
Root-cause investigation	No	No	Yes - 2-hop BFS + LLM reasoning
Ordered repair planning	No	No	Yes - topological sort on affected-file graph
Real test execution	No	No	Yes - pytest + bandit in subprocess sandbox
Security re-verification	No	No	Yes - targeted re-scan post-fix
Confidence scoring	No	No	Yes - 5-signal composite, 0-100%
Human approval gate	No	No	Yes - auto-triggered on low confidence or critical path
Repository memory	No	No	Yes - ChromaDB, shared across repos
LLM Orchestrator	No	No	Yes - adaptive, handles partial failures

10. Limitations & Future Work

Test execution requires the target repo to have a working test suite; without tests the Validation Agent falls back to static analysis only, lowering the maximum achievable confidence score. The AST knowledge graph currently handles direct imports reliably but can miss heavily dynamic patterns (decorators, monkey-patching); ambiguous edges conservatively widen the blast radius. ChromaDB memory improves with use - early runs on a fresh instance have no cache hits. Planned: sandboxed Docker execution for full CI parity, PR auto-merge for confidence $\geq 95\%$, and GPT-4o as a secondary LLM for cost optimisation on triage calls.